The Project Report

NAME : A.KAMAL SIDDARTHA

REGISTRATION NUMBER : 24BAI10185

DATE OF SUBMISSION : 27-03-2026

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve:

Before Git existed, the Linux kernel development team faced a major crisis in 2005. For three years (2002–2005), they had been using BitKeeper, a proprietary distributed version control system (DVCS) provided free of charge by BitMover. BitKeeper worked well for a large, geographically distributed team like the kernel developers because it allowed offline work, fast branching, and efficient merging of thousands of patches.

The crisis hit when a kernel developer reverse-engineered the BitKeeper protocol to create an open-source client. BitMover responded by revoking the free license for the entire Linux community, including Linus Torvalds. This left the team without a suitable tool. Traditional options like CVS or early Subversion were centralized, slow for large projects, and poor at handling non-linear development and massive patch sets. Linus explicitly refused to use any proprietary tool again, stating that the kernel's source code must remain completely free and open.

In just 10 days (April 2005), Linus wrote the initial version of Git from scratch. His goal was not just to replace BitKeeper but to create something better: a fast, simple, fully distributed system optimized for the Linux kernel's scale (millions of lines of code and thousands of contributors). Git's design deliberately avoided BitKeeper's weaknesses while incorporating lessons from it — cryptographic integrity via SHA-1 hashes, contentaddressable storage, and cheap branching/merging. Within weeks, Git was self-hosting the Linux kernel. This origin story shows open source's power: when proprietary tools fail the community, developers build their own free alternative. Git was born out of necessity and a strong commitment to freedom.

A2. The license — what it actually says

I downloaded the exact license from Git's official repository. It is the GNU General Public License version 2 (GPL v2).

- **The four freedoms of free software:**
    1. Freedom to run the program for any purpose.
    2. Freedom to study and modify the source code.
    3. Freedom to redistribute copies.
    4. Freedom to distribute modified versions. **Git's GPL v2 license grants all four freedoms.**

- **Can a company take this software, modify it, and sell it without sharing changes?** No. GPL v2 is a **copyleft** (or "share-alike") license. Any modified version distributed to others must also be released under GPL v2 with full source code. This prevents proprietary forks.

- **Difference between GPL and MIT licensing? Which would I choose?** GPL is copyleft — it forces derivatives to remain free and open. MIT is permissive — you can take the code, modify it, and release a closed-source commercial product without sharing changes. For something I build, I would choose **MIT**. It allows wider adoption (companies can use it in proprietary products), which encourages more contributions and real-world use. GPL is ideal when the goal is to protect user freedom forever, but MIT better balances freedom with practicality for many modern tools.

- **Free as in free beer vs. free as in freedom?** "Free as in free beer" means no cost to obtain (gratis). "Free as in freedom" means liberty to use, study, modify, and share (libre). Git is both: you download it for free and have full freedom to inspect and change every line. Example: I can fork Git, add a new feature, and share my version — no company can stop me.

A3. The ethics of open source :

Open source is more than a license — it is a philosophy of sharing knowledge.

- Should all software be open source?For: Transparency leads to better security (anyone can audit code), faster innovation (thousands of eyes improve it), and equity (students and small teams in developing countries get world-class tools). Against: Some software (e.g., proprietary encryption algorithms or competitive business logic) needs secrecy to protect intellectual property and incentivize investment. Forcing everything open could reduce funding for specialized commercial tools. My view: Not *all* software, but critical infrastructure (OS kernels, browsers, databases) should be open.

- Is it ethical for a company to build a profitable product entirely on top of free community labor? Yes, if they contribute back. Companies like Red Hat, Google, and Meta heavily use Linux and Git but also sponsor developers, fix bugs, and release tools (e.g., Google's contributions to Git). This creates a virtuous cycle: the community gets better software, and companies get stable foundations. Pure "take without giving" would be unethical.

- What does 'standing on the shoulders of giants' mean in software? It means every project builds on previous open work. Git stands on the shoulders of BitKeeper's ideas (without copying code) and earlier tools like RCS. Open source accelerates innovation because developers don't reinvent wheels — they improve existing ones. It does *not* undermine originality; it multiplies it. Without open source, modern software development would be decades behind.

Part B — Linux Footprint (Unit 2)

I installed Git on a Linux VM (Ubuntu 24.04 LTS, but scripts were tested on an RPM-based system as per starter templates).

- **Installation method:** On Debian/Ubuntu: sudo apt install git. On RPM-based (Fedora/RHEL): sudo dnf install git (or yum). It is also easy to compile from source.

- **Key directories:**
  - Binaries: /usr/bin/git
  - System config: /etc/gitconfig
  - User config: ~/.gitconfig
  - Per-repository: .git/ (inside every repo)
  - Logs: Git itself does not produce system logs; repo history is in .git/logs/.

- **User and group:** Runs as the current logged-in user (no dedicated service user). This matters for security — permissions are file-based, preventing privilege escalation.

- **Service management:** Git is a command-line tool, not a daemon. No systemctl commands needed.

- **Update model:** Security patches come through the Linux distribution's package manager (apt/dnf). The open-source community reviews and merges fixes in the official Git repository; distros package and push them quickly.

Part C — The FOSS Ecosystem (Units 3 & 4)

Git does not have heavy runtime dependencies (written in C with minimal libraries like zlib). Build-time dependencies include GCC, Make, and OpenSSL (optional).

It has enabled an entire industry: GitHub, GitLab, Gitea, CI/CD tools (Jenkins, GitHub Actions), and almost every modern open-source project. Forks include libgit2 and JGit (Java implementation).

In the LAMP stack, Git versions the PHP/Apache/MySQL codebases developers work on. It relates to the web by powering platforms that host billions of repositories.

Community: Governed by a core team of maintainers led by Linus Torvalds (benevolent dictator for life style, though now more collaborative). Main communication is the git@vger.kernel.org mailing list. There are no formal conferences owned by a single entity, but events like Git Merge exist. No single company controls it — it is truly communitydriven.

Part D — Open Source vs Proprietary (Critical Analysis)

Overall verdict: I strongly recommend Git for any real-world deployment. It is faster, more reliable, and more flexible than proprietary alternatives for 99% of projects. I would happily contribute to Git — in fact, its model inspired me to start using version control properly in my own projects. Proprietary tools like Perforce excel only in very specific large-scale binary-heavy environments (e.g., game development), but Git's ecosystem has largely overtaken them.

Dimension	**Open Source (Git)**	**Proprietary Alternative (Perforce / Helix Core)**
Cost	Completely free	Expensive per-user licenses
Security (who can audit the code?)	Anyone in the world can audit and fix	Only company employees; black-box
Support and reliability	Community + enterprise options (GitLab, etc.)	Paid professional support with SLAs
Freedom to modify	Full freedom; fork anytime	Restricted; vendor controls changes
Community vs corporate control	Community-driven; no single owner	Corporate-controlled; vendor decides roadmap
Your overall verdict	—	—

1. Shell Script Tasks

All five scripts are included below with code, explanation, and the required concepts. Each was tested on Linux and includes comments.

Script 1 — System Identity Report (Unit 1 & 2

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Sathvik | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Sathvik"
SOFTWARE_CHOICE="Git"

# --- System info --- KERNEL=$(uname
-r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d"" -f2)
DATE_TIME=$(date '+%Y-%m-%d %H:%M:%S')
LICENSE_MSG="The Linux OS is covered under the GNU General Public License (GPL)."

# --- Display ---
echo   "================================="
echo "  Open Source Audit — $STUDENT_NAME"
echo "  Chosen Software: $SOFTWARE_CHOICE"
echo "================================="
echo "Kernel     : $KERNEL" echo "User       :
$USER_NAME" echo "Distro     : $DISTRO"
echo "Uptime     : $UPTIME" echo "Date/Time  :
$DATE_TIME" echo "OS License :
$LICENSE_MSG" echo
"================================="
```

```
=================================
  Open Source Audit — Sathvik
  Chosen Software: Git
=================================
Kernel     : 4.4.0
User       : root
Distro     : Ubuntu 24.04.2 LTS
Uptime     : up 0 minutes
Date/Time  : 2026-03-27 09:50:38
OS License : The Linux OS is covered under the GNU General Public License (GPL).
=================================
```

Explaination : Uses variables, command substitution $( ), and echo for formatted output. Demonstrates basic Linux info gathering (concepts from Unit 1 & 2).

Script 2 — FOSS Package Inspector (Unit 2)

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"   # Chosen software

# Check if package is installed (works on RPM or Debian; simplified for demo)
if command -v rpm &> /dev/null && rpm -q $PACKAGE &>/dev/null; then
echo "$PACKAGE is installed."     rpm -qi $PACKAGE | grep -E
'Version|License|Summary' elif command -v dpkg &> /dev/null && dpkg -l |
grep -q $PACKAGE; then     echo "$PACKAGE is installed."     dpkg -l | grep
$PACKAGE else     echo "$PACKAGE is NOT installed." fi

# Case statement with philosophy note case
$PACKAGE in
   git)    echo "Git: the tool Linus built when proprietary failed him — distributed, free, and
revolutionary." ;;     httpd)  echo "Apache: the web server that built the open internet" ;;
mysql)  echo "MySQL: open source at the heart of millions of apps" ;;     vlc)    echo
"VLC: plays anything — built by students for freedom" ;;     *)      echo "Open-source tool
enabling collaboration and freedom." ;; esac.
```

Explaination : Uses if-then-else, case statement, pipes with grep, and package queries (Unit 2).

Script 3 — Disk and Permission Auditor (Unit 2)

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report" echo
"----------------------"

for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
       PERMS=$(ls  -ld  "$DIR"  |  awk  '{print  $1,  $3,  $4}')
SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)       echo "$DIR
=> Permissions: $PERMS | Size: $SIZE"     else
```

```
        echo "$DIR does not exist on this system"
fi
done

# Check Git config (as per TODO)
GIT_CONFIG_DIR="$HOME/.gitconfig" if
[ -f "$GIT_CONFIG_DIR" ]; then
    PERMS=$(ls -l "$GIT_CONFIG_DIR" | awk '{print $1, $3, $4}')
echo "Git user config => Permissions: $PERMS" else
    echo "Git user config not found — run 'git config --global' to create."
fi
```

Explanation: Uses for loop, ls -ld, du, awk, and conditional checks (Unit 2).

Script 4 — Log File Analyzer (Units 2 & 5)

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog error

LOGFILE=${1:-"/var/log/syslog"}
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE" echo
"Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Explanation : Uses while read loop, if-then, counter, command-line arguments $1, and tail (Units 2 & 5).

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
```

```
echo "Answer three questions to generate your manifesto." echo
""

read -p "1. Name one open-source tool you use every day: " TOOL read
-p "2. In one word, what does 'freedom' mean to you? " FREEDOM read
-p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

cat > "$OUTPUT" << EOF Open
Source Manifesto — $DATE

I, Sathvik, believe in open source because tools like $TOOL have taught me that $FREEDOM is the
foundation of innovation.
I would build and share $BUILD freely so others can stand on my shoulders and create even greater things.
This is the spirit of Git and the entire FOSS movement.


— Sathvik
EOF

echo "Manifesto saved to $OUTPUT"
cat "$OUTPUT"
```

Explanation: Uses read for interactive input, string variables, cat > redirection to file, and date command (Unit 5). Alias concept shown in comment style

Conclusion and Final Verdict

The transition of the Linux Kernel project from a proprietary tool like BitKeeper to the community-driven Git remains one of the most significant moments in open-source history . Through this audit, it is clear that Git is not merely a utility but a manifestation of the "Free as in Freedom" philosophy . While proprietary alternatives like Perforce may offer specialized features for massive binary files, Git’s speed, cryptographic integrity, and decentralized nature make it the superior choice for modern, non-linear development . I strongly recommend Git for any real-world deployment, as its massive ecosystem ensures that a solution is always available for any technical challenge .

On a personal level, this project has deeply shifted my perspective on software development. Writing shell scripts to automate system tasks showed me how the transparency of open source allows a student to "stand on the shoulders of giants" to build meaningful tools . I am now fully committed to using Git for all my future engineering projects and intend to contribute back to the FOSS community. This audit has proven that the true value of open source lies not in the "free price tag," but in the collective power of a global community working toward a common goal of innovation and liberty .